Fault-Tolerant Distributed Systems

Lab 2 – Quorums

2026/2027

Deliverable: lab record, due by the end of Friday, 13 November (see the last section).

Goal Lab 1 ended with a question: is there a middle ground between waiting for one server and waiting for all of them? In this lab you build a key-value store in which every operation waits for *some* of the servers, find out what it needs in order to be correct, and measure how many failures it tolerates.

Learning outcomes Choose read and write quorums from intersection conditions. Explain why replicas must agree on an order of writes. Distinguish what quorums guarantee for completed writes from what they guarantee for writes still in progress. Measure the availability of a quorum configuration. Explain why a read-modify-write operation needs more than quorums.

How we work As in Lab 1: think first, individually and then as a group, and only then implement and run. Follow the AI workflow of Lab 1 (Part 2), with one addition: *implement exactly the design of the current part*. If the assistant proposes mechanisms you have not decided on, write them down in your AI log and reject them for now; you may need them later.

Setup Download the new `kit.tar.bz2` from eLearning and extract it over your Lab 1 kit (your own files are kept). It adds `summary.py`; read its entry in `kit/README.md`. Your programs read the quorum sizes from the environment, so that the same file covers every configuration, e.g. for $Q_r = 1$ and $Q_w = 3$:

```
QR=1 QW=3 ./run.sh lin-kv q.py
```

Reading the results in this lab

- Until Part 4 your store answers `cas` with error 10 (not supported). Maelstrom then reports *Analysis invalid!* on *every* run, because one operation type never succeeds. The verdict that matters is linearizability: run `./summary.py`, which prints it (`lin=`) with the operation counts.
- For the same reason `--test-count` stops after the first run. To repeat a run, use a loop, then summarize all runs:

```
for i in 1 2 3; do
  QR=2 QW=2 ./run.sh lin-kv q.py --latency 50 --rate 50
done
./summary.py store/lin-kv/2*
```

- A `read` that times out counts as *fail*, not *info*: it had no effect. Unavailable writes count as *info*.

Part 0 – Predict *(10 min)*

Individually, without discussing and without AI, write down short answers:

1. With $N = 3$ servers, list every pair (Q_r, Q_w). For which pairs must a `read` that starts after a `write` has completed see that write? Why?
2. A reader gets different values from different servers. How should it decide which one to return?
3. With $N = 5$, how many crashed servers can majority quorums tolerate? And read-one/write-all (ROWA)? Answer for reads and writes separately.

Part 1 – Wait for some *(40 min)*

The design for this part:

- Each server stores, for each key, only its value.
- `write`: the receiving server sends the new value to all servers (itself included) and replies to the client after Q_w of them have acknowledged it.
- `read`: the receiving server asks all servers for their value and replies after receiving Q_r answers.
- `cas`: reply with error 10 (not supported).

1. As a group, decide what the reader returns when the Q_r answers disagree.
2. *Predict*, for each configuration and scenario below, whether the run will be linearizable and whether clients will be able to complete operations. Justify each prediction in one sentence.

Configuration	(Q_r, Q_w)	Scenario	Command options
ROWA	$(1, 3)$	No faults	(none)
Majority	$(2, 2)$	Latency, more load	`--latency 50 --rate 50`
Non-intersecting	$(1, 2)$	A server crashes	`MS_CRASH="n1@5"`
		Network partitions	`--nemesis partition`

3. Implement the design (`q.py`), following the AI workflow.
4. Run all twelve combinations and record the results next to your predictions. One scenario for the three configurations at a time (about two minutes); discuss your predictions while it runs:

```
for q in "1 3" "2 2" "1 2"; do set -- $q
  QR=$1 QW=$2 ./run.sh lin-kv q.py --latency 50 --rate 50 &>/dev/null
  echo "QR=$1 QW=$2 $(./summary.py)"
done
```

5. Explain one non-linearizable run in detail, with the configuration assigned to your group: majority (groups 1, 4, 7, 10), ROWA (2, 5, 8), non-intersecting (3, 6, 9). Which choice of the design is to blame: what the servers store, or what the reader returns? ROWA groups: compare with strategy B of Lab 1. Non-intersecting groups: would any change to what the servers store fix your run? If your configuration produced no such run, explain why not, and which scenario could produce one.

From now on, everyone uses majority quorums, $Q_r = Q_w = 2$ with $N = 3$.

Part 2 – Agree on the newest *(25 min)*

1. As a group, without a computer: what extra information would let the servers, and the reader, agree on which of two values is the newest? Who produces this information, and how? Draw a space-time diagram of two writes followed by a read with your design.
2. Implement it. Add `trace()` calls where the information is created and where it is compared.
3. Run the scenarios of Part 1. Repeat `--latency 50 --rate 50` at least three times; if all runs are linearizable, increase `--rate` and `--latency`. When a run is not linearizable, find the operations involved with `./timeline.py --history --key 0`, and write down your explanation before moving on.

Checkpoint: groups that made different choices in question 1 will be asked to compare their runs.

Part 3 – Concurrent writers, and the limits *(25 min)*

Optional, before starting: ask an AI assistant whether your Part 2 store is linearizable, and why. Compare its answer with what you find in this part, and add both to your AI log.

1. Using your traces, find two writes on the same key that ran concurrently. What did each server end up storing, and why? What rule would make all servers agree, without any extra messages?

2. Implement the rule and repeat the runs under latency and load.
3. *Availability.* With the protocol you now have and `--node-count 5`, run majority $(3, 3)$, ROWA $(1, 5)$, and one configuration of your choice, under four scenarios: no faults, `MS_CRASH="n1@5"`, `MS_CRASH="n1@5,n2@5"`, and `--nemesis partition`. Record the verdict and the operation counts, and compare them with your answer to Part 0, question 3. Note that each client always talks to the same server: clients of a crashed server lose their operations whatever the protocol. Also record `msgs/op`: why is it almost the same for every configuration?
4. *Predict, without a computer.* $N = 3$, majority quorums. A write has reached only the server that received it. Draw an execution in which two reads that do not overlap return, first, the new value and then the old one. Is it possible with your protocol?
5. Such executions are rare with majority quorums. Download `inversion-run.tar.bz2` from eLearning and extract it in `kit`: it is a run of a protocol like yours, in which one link was made artificially slow (see the `slow_put` events). Find the execution you drew:

   ```
   ./timeline.py inversion-run --history --key 0 | grep -v cas
   ```

   Did any of your ROWA runs of question 3 show the same pattern?

Part 4 – Compare-and-set *(30 min)*

1. Implement `cas` on your store: read from a quorum, compare, then write as a `write` would. Add a `trace()` with what the `cas` read.
2. *Predict, then run* with `--latency 50 --rate 50`. Using your traces, find two `cas` on the same key that both succeeded although only one should have. Why is this impossible with a single server? (The minimal history shown by the checker may not include the two `cas`: use your traces.)
3. As a group, without a computer, design a lock: before a `cas`, the receiving server locks the key at a quorum of servers, and unlocks it when done.
   - How large must the lock quorum be so that two operations can never hold the lock at the same time? Relate the answer to Part 0.
   - What does a server do when asked to lock a key that is already locked? What does the requester do then?
   - Do `write` operations also need the lock? Why?
4. Implement the lock and run with `--latency 50 --rate 50`, then add `MS_CRASH="n1@5"`, then try `--nemesis partition`. Record the counts. What happens to the other clients when the server that holds a lock crashes? When two servers each lock part of the quorum?

Part 5 – Explain one execution *(20 min)*

Choose the most interesting run you obtained: a lost update or a blocked key from Part 4, or one of yours from Part 3. If you have none, use `inversion-run`.

1. Using `linear.svg`, `messages.svg`, `timeline.html` and `./timeline.py --history`, reconstruct the execution: which clients, servers and messages were involved, and in which order.
2. Draw it as a space-time diagram and mark the point where things went wrong.
3. Restricted to one key, is the history linearizable? Is it sequentially consistent? Justify both answers with the definitions. Would your answers change if the operations involved had been performed by the same client?
4. Revise your claim from Lab 1 for the store you now have: under which assumptions, how many failures of which kind, which guarantee, with which evidence, and what is not covered.

Part 6 – Compare across groups *(5 min)*

Each group adds its Part 1 results to the class table (configuration, scenario, predicted, observed). Then discuss:

1. Did all groups observe the same results for the same configuration and scenario? Where they did not, why?
2. A server is waiting for a lock whose holder may have crashed. How could it take the lock over safely, without ever having two holders?

Going further (optional)

Change the `read` so that the execution of Part 3, question 4, becomes impossible. Explain why it does, and show it with runs (e.g. with ROWA and partitions, as in Part 3, question 3). What does it cost? If you do it, include it in your record: as a step of the design evolution, with its runs as evidence.

Exit record *(5 min)*

Each group writes down, in four lines: (1) the claim considered; (2) the strongest evidence obtained; (3) one limitation or unresolved question; (4) the next action.

Deliverable: lab record

By the end of Friday, 13 November, each group submits on eLearning a lab record of at most three pages, using the template `record.tex` in the kit, together with the code and the `store/` directories of the runs it cites (only those). Finish at home what you did not complete in class. For this lab:

1. **Claim**: your revised claim from Part 5.
2. **Design and predictions**: instead of a single table:
   - the evolution of your design, one line per part, stating what broke, what you added and why, citing the run that showed it;
   - the table of Part 1 (predicted and observed);
   - the table of Part 3, question 3, and the configuration you would choose for a read-heavy and for a write-heavy service, with a justification.
3. **Evidence**: the execution of Part 5 as an annotated space-time diagram, with its classification.
4. **Limitations and open questions**: what your runs do not show, and what you would test next.
5. **AI log**: the prompts that mattered and what you had to correct or reject, including any mechanism the assistant proposed before you needed it.
6. **Contributions**: one line per member.

Lab records are kept as a portfolio and are the basis of the final discussion of the practical work, in which each member is asked individually to explain (or extend) what the group did.